

CHAPTER 1

INTRODUCTION

1.1 Overview

Artificial Intelligence (AI) and Machine Learning (ML) have transitioned from localized, task-specific pattern recognition algorithms into foundation-scale systems capable of advanced semantic understanding, natural language reasoning, and complex contextual evaluation. Within this landscape, Large Language Models (LLMs) and Generative AI represent a substantial paradigm shift. Trained on extensive cross-domain textual corpora, LLMs exhibit strong emergent capabilities in contextual synthesis, structured critique, multi-factor comparison, and natural language communication. Consequently, AI-based decision-making systems are increasingly explored across institutional environments—such as banking credit evaluations, employment candidate screening, and university academic admissions—to automate high-volume evaluations and support human decision-makers.

Despite their reasoning capabilities, conventional AI-based decision systems face acute transparency, reliability, and interpretability challenges. High-capacity statistical classifiers, deep neural networks, and standalone generative LLMs predominantly operate as monolithic or opaque decision units. When a single model is prompted to directly output a verdict (e.g., whether an applicant should be approved or rejected), several vulnerabilities emerge. A single model inference path often anchors prematurely on prominent applicant attributes while failing to systematically challenge its own preliminary assumptions. Explanations generated in a single pass frequently function as post-hoc rationalizations constructed to justify an already chosen verdict, rather than reflecting a balanced, dialectical evaluation of competing factors. Furthermore, without explicit, decoupled evaluation stages, single-pass models can overlook regulatory constraints, fail to cross-examine conflicting self-reported claims, or omit crucial cautionary counter-evidence.

To address these limitations, the field of Explainable Artificial Intelligence (XAI) emphasizes systems whose internal logic, deliberation dynamics, and evidence weighting are accessible, auditable, and interpretable by human reviewers. Multi-Agent Systems (MAS) and LLM-based autonomous agents offer an architecturally grounded solution to transparency. Rather than relying on a monolithic inference call, multi-agent frameworks decompose a complex evaluative challenge into specialized, modular agent roles operating under explicit behavioral directives.

The proposed Multi-Agent LLM-Based Transparent Decision-Making System is designed as a web-based decision-support platform built on the Python Flask framework and powered by the

Google Gemini API. The system models a rigorous institutional review committee by organizing four distinct, specialized LLM agent personas into a deterministic evaluative pipeline:

Advocate Agent: Examines submitted applicant attributes to identify positive merits, qualifications, financial strengths, or academic achievements, constructing the strongest possible affirmative case.

Skeptic Agent: Scrutinizes the application for financial liabilities, academic discrepancies, risk exposures, capacity constraints, or unverified claims, formulating critical counterarguments.

Compliance Checker Agent: Evaluates the profile against established institutional criteria, regulatory guidelines, policy thresholds, and categorical eligibility prerequisites.

Adjudicator Agent: Receives the complete applicant profile alongside the accumulated outputs of the Advocate, Skeptic, and Compliance Checker. It weighs the conflicting arguments, deliberates across the multi-perspective evidence, and synthesizes a final binary decision (APPROVED or REJECTED) accompanied by a transparent, structured explanation.

The application serves strictly as an assistive decision-support system. The project does not train or fine-tune Gemini models from scratch; instead, it leverages the Google Gemini API using structured prompt engineering, system-level role conditioning, and deterministic multi-agent pipeline orchestration within a lightweight Flask application.

1.2 Problem Statement

Conventional automated decision-making and screening platforms suffer from fundamental architectural constraints that hinder their adoption in high-stakes domains:

Single-Perspective Inference: Standard machine learning classifiers and monolithic LLM prompts evaluate complex profiles along a single, unified computation path. This lack of role diversity makes them vulnerable to confirmation bias and premature convergence.

Opaque and Post-Hoc Explanations: Most automated systems produce either non-interpretable numerical scores (e.g., credit scores, probability thresholds) or single-pass textual summaries that fail to articulate the competing trade-offs considered during evaluation.

Rigid Rule Structures vs. Narrative Nuance: Conventional rule-based systems apply inflexible boolean logic that fails to interpret contextual, qualitative nuances (such as non-traditional academic trajectories or transient financial anomalies). Conversely, unstructured single-pass models struggle to reliably adhere to strict institutional guidelines.

Failure to Detect Conflicting Evidence: In high-volume screening, applications frequently contain subtle tensions between quantitative metrics and qualitative statements. Monolithic architectures lack an adversarial mechanism to actively surface, dissect, and reconcile such discrepancies.

Absence of Structured Deliberation and Decoupled Policy Auditing: Real-world institutional decision-making relies on committee debates where advocacy, risk scrutiny, and regulatory compliance are handled by distinct reviewers. Existing automated systems rarely provide this separation of concerns.

Formal Problem Statement: To design, implement, and evaluate a modular, transparent, web-based multi-agent decision-support architecture using the Flask framework and the Google Gemini API, wherein applicant data is sequentially evaluated by specialized Advocate, Skeptic, and Compliance Checker agents before being deliberated and decided by an Adjudicator Agent, thereby producing auditable binary outcomes supported by balanced, multi-perspective explanatory rationales.

1.3 Research Motivation

High-stakes administrative and screening evaluations—such as consumer credit qualification and university admissions—demand both analytical rigor and ethical defensibility. When applicants are subject to automated screening, opacity undermines user trust, complicates administrative auditing, and risks perpetuating unexamined systemic biases. The primary motivation of this research is to replace opaque black-box decision routines with an interpretable, dialectical framework inspired by human committee review. By decomposing the decision process into explicit, adversarial, and complementary roles, the Advocate guarantees that merit and potential are actively articulated; the Skeptic ensures that liabilities, risks, and vulnerabilities are not minimized; the Compliance Checker provides a dedicated filter against categorical policy violations; and the Adjudicator operates with full access to these conflicting perspectives, mirroring judicial deliberation.

Structuring the decision-support workflow through role-conditioned prompts provides human operators with an auditable deliberation trace. Rather than inspecting an isolated final verdict, human reviewers can inspect the affirmative merits, risk factors, and policy findings independently. While multi-agent deliberation does not provide a mathematical guarantee of decision optimality or eliminate all underlying model biases, it establishes an explicit framework for transparent, accountable, and explainable decision support.

1.4 Research Objectives

The primary objectives established for this project include:

Develop a Modular Web Application: Implement a lightweight, responsive web application using the Python Flask framework to handle applicant data intake and presentation.

Integrate the Google Gemini API: Establish robust API connectivity with Google Gemini using secure environment variable configuration and the official client SDK.

Implement the Advocate Agent: Construct role-specific prompt templates instructing Gemini to extract affirmative arguments, strengths, and supporting qualifications from submitted applicant data.

Implement the Skeptic Agent: Develop adversarial prompt templates directing Gemini to identify risk factors, potential liabilities, data inconsistencies, and negative indicators.

Implement the Compliance Checker Agent: Design compliance evaluation prompts that verify the applicant's profile against domain-specific eligibility thresholds, prerequisite rules, and regulatory considerations.

Implement the Adjudicator Agent: Construct an adjudicative prompt template capable of synthesizing the raw application data, Advocate arguments, Skeptic critiques, and Compliance findings into a final balanced judgment.

Orchestrate Deterministic Sequential Execution: Chain the agent invocations within the Flask backend such that downstream agents receive contextualized outputs from upstream agents in a verified sequence.

Generate Structured Binary Outputs: Ensure the Adjudicator Agent produces unambiguous binary decision outputs (APPROVED or REJECTED) conforming to expected application formats.

Deliver Human-Readable Explanations: Format and display the multi-agent deliberation trail, providing human operators with an interpretable breakdown of merits, risks, compliance status, and final rationale.

Ensure Domain-Agnostic Extensibility: Validate the modularity of the multi-agent framework across two distinct evaluation domains: loan pre-screening and academic admissions support.

1.5 ignificance

The project possesses multidimensional significance across computer science and applied artificial intelligence. Academically, it operationalizes theoretical literature on multi-agent debate and role prompting into an applied software engineering architecture. Technically, it illustrates how modern generative APIs can be orchestrated within lightweight web microframeworks without requiring expensive dedicated GPU infrastructure. From an Explainable AI (XAI) standpoint, it shifts explainability from post-hoc mathematical approximations toward intrinsic procedural transparency, where the explanation is formed by the debate itself. Finally, from a Responsible AI perspective, decoupled compliance and adversarial critique layers serve as structural safety gates that prevent unexamined assumptions from dominating automated screening.

1.6 Advantages

Multi-Perspective Deliberation: Prevents single-pass cognitive anchoring by enforcing independent affirmative and skeptical evaluations.

Procedural Transparency: Human auditors can inspect the precise textual arguments generated at each stage of the multi-agent pipeline.

Explicit Risk Surfacing: Dedicated skeptical analysis actively searches for liabilities, debt ratios, or academic deficits that might be overlooked in generalized scoring.

Dedicated Compliance Isolation: Decouples policy verification from general qualification, ensuring institutional rules receive distinct analytical focus.

System Modularity: Agent personas, prompt instructions, and evaluation criteria can be independently modified or reconfigured without rewriting the core backend.

Extensibility across Domains: Readily adapts to varied evaluative contexts by modifying prompt definitions.

Human-Readable Reporting: Generates natural-language syntheses readily understood by non-technical administrative personnel and applicants.

(Note: The system provides structured analytical perspectives but does not claim mathematically guaranteed accuracy, absolute fairness, or complete elimination of algorithmic bias.)

1.7 Applications

Loan Application Pre-Screening: Assisting credit officers by providing balanced summaries of financial capacity, income stability, credit liabilities, and lending policy conformity.

Academic Admission Support: Supporting university admissions committees by parsing academic transcripts, extracurricular achievements, prerequisite compliance, and potential academic risk indicators.

Scholarship Evaluation Screening: Reviewing merit-cum-means criteria, academic standing, and socio-economic support claims for institutional grant committees.

Employment Candidate Pre-Screening: Analyzing candidate resumes against job requisites, identifying career strengths alongside potential skill gaps or experience deficiencies.

Insurance Application Intake Analysis: Assisting underwriters in pre-evaluating risk disclosures, policy coverage criteria, and applicant background histories.

Regulatory and Eligibility Screening: Conducting preliminary reviews of public sector grant applications, permit submissions, or licensing requests against statutory requirements.

Crucial Operational Notice: In all deployable contexts, the system operates exclusively as a decision-support mechanism. High-impact decisions directly affecting human livelihoods, legal

standing, financial rights, or educational opportunities strictly require authoritative human oversight and professional validation.

CHAPTER 2

LITERATURE SURVEY

2.1 Survey of Related Literature

The convergence of Large Language Models, Multi-Agent Systems, and Explainable AI has stimulated extensive scholarly research into automated deliberation, collaborative reasoning, and algorithmic transparency. The following points summarize verified benchmark contributions informing this project.

1.Improving Factuality and Reasoning in Language Models through Multiagent Debate

Authors: Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, Igor Mordatch

Year & Venue: 2024 ICML

Methodology & Contribution: Engaged multiple LLM instances in iterative debate. Showed multi-agent debate consistently improves factual validity and mathematical reasoning over single models.

Limitation: High token overhead in unstructured debate.

Relevance: Theoretical basis for competing Advocate vs. Skeptic agents.

2. ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate

Authors: Chi-Min Chan, Weize Chen, Yusheng Su, Jianxuan Yu, Wei Xue, Shanghang Zhang, Jie Fu, Zhiyuan Liu

Year & Venue: 2024 ICLR

Methodology & Contribution: Multi-agent role-playing framework where diverse LLM personas debate text quality, mirroring human panel evaluation.

Limitation: Focused on open-ended text quality rather than administrative decisions.

Relevance: Inspires panel-style evaluation for applicant screening.

3. CAMEL: Communicative Agents for 'Mind' Exploration of Large Language Model Society

Authors: Guohao Li, Hasan A. K. Hammoud, Hani Itani, Dmitrii Khizbullin, Bernard Ghanem

Year & Venue: 2023 NeurIPS

Methodology & Contribution: Role-playing conversational framework using inception prompting to facilitate autonomous cooperation toward common tasks.

Limitation: Unconstrained agents can suffer role drift.

Relevance: Guided prompt engineering and strict role-conditioning for specialized agents.

4. Explainability for Large Language Models: A Survey

Authors: Haiyan Zhao, Hanjie Chen, Fan Yang, Ninghao Liu, Huiqi Deng, Hengyi Cai, Shuaiqiang Wang, Dawei Yin, Mengnan Du

Year & Venue: 2023 ACM TIST / arXiv

Methodology & Contribution: Comprehensive taxonomy of LLM interpretability and prompt-based explainability techniques across prompting paradigms.

Limitation: Single-pass explanations risk post-hoc rationalization.

Relevance: Establishes necessity of multi-perspective deliberation traces.

5. A Survey on Large Language Model based Autonomous Agents

Authors: Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, et al.

Year & Venue: 2024 Front. Comput. Sci.

Methodology & Contribution: Systematic taxonomy of LLM agent architectures, profiling strategies, planning algorithms, and cooperative multi-agent dynamics.

Limitation: Focuses on theoretical taxonomies rather than web decision systems.

Relevance: Architectural grounding for profiling distinct agent roles.

6. AI safety via debate

Authors: Geoffrey Irving, Paul Christiano, Dario Amodei

Year & Venue: 2018 arXiv

Methodology & Contribution: Theoretical framing of zero-sum debate games between AI agents judged by an adjudicator, proving debate reveals truth more effectively.

Limitation: Assumes idealized judge discernment.

Relevance: Foundational justification for pairing Advocate against adversarial Skeptic.

7. A Survey on Bias and Fairness in Machine Learning

Authors: Ninareh Mehrabi, Fred Morstatter, Nripsuta Saxena, Kristina Lerman, Aram Galstyan

Year & Venue: 2021 ACM Comput. Surv.

Methodology & Contribution: Documented how representation and systemic biases enter algorithmic pipelines and distort automated decision systems.

Limitation: Emphasized classical ML classifiers.

Relevance: Underscores necessity of explicit compliance checks and human oversight.

8. Guidelines for Human-AI Interaction

Authors: Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, et al.

Year & Venue: 2019 ACM CHI

Methodology & Contribution: Established 18 core design heuristics for transparent human-AI interaction, highlighting procedural clarity and human control.

Limitation: Formulated prior to modern generative LLM workflows.

Relevance: Guides UI design to present granular deliberation cards to human users.

2.2 Detailed Analytical Review of Literature

The surveyed literature establishes that while foundation models demonstrate advanced semantic reasoning, individual models frequently falter on complex evaluative tasks due to cognitive blind spots, sycophancy, and uncritical pattern matching. Du et al. (2024) systematically demonstrated that engaging multiple language model instances in structured debate consistently outperforms single-model baselines across factual validation and deductive reasoning benchmarks. By compelling individual instances to inspect opposing arguments, the system surfaces errors and ungrounded assumptions that persist under single-model chain-of-thought prompting. Complementing this, Chan et al. (2024) introduced ChatEval, demonstrating that role-conditioned multi-agent panels evaluate qualitative artifacts with significantly higher alignment to expert human assessment than monolithic evaluators.

In the realm of autonomous agent design, Li et al. (2023) explored CAMEL, proving that inception prompting and strict persona specification enable autonomous agents to execute complex cooperative workflows without human intervention. Wang et al. (2024) synthesized these paradigms into a comprehensive architectural survey, establishing that multi-agent pipelines benefit from decoupled role specialization, where each sub-agent focuses on a discrete cognitive responsibility. The fundamental safety and transparency advantages of adversarial deliberation trace back to the theoretical framework of Irving, Christiano, and Amodei (2018). Their research showed that framing automated evaluation as a debate between competing agents assists the adjudicator in uncovering subtle fallacies, since neither agent can conceal adverse facts without the opposing agent immediately exposing them.

However, as highlighted by Zhao et al. (2023), natural-language explanations generated by generative models are susceptible to hallucinated rationalization if they are merely generated after an unconstrained decision has been made. Furthermore, Mehrabi et al. (2021) demonstrated that algorithmic screening tools risk amplifying systemic bias if policy criteria are implicit rather than rigorously audited. Finally, Amershi et al. (2019) underscored that real-world AI decision systems must maintain strict operational humility, clearly presenting system rationale to ensure meaningful human-in-the-loop oversight.

2.3 Research Gap Analysis

While existing literature validates the theoretical benefits of multi-agent debate and agentic role-playing, significant gaps remain between theoretical consensus frameworks and practical, lightweight decision-support implementations:

Focus on Open-Ended Benchmarks vs. Administrative Decision Support: Most multi-agent debate studies focus on arithmetic puzzles, competitive games, or open-ended text quality scoring, leaving structured administrative screening tasks (such as credit evaluation or admissions) largely unexamined.

Absence of Dedicated Regulatory/Compliance Isolation: Existing multi-agent debate frameworks primarily pit affirmative advocates against negative critics. They fail to integrate an independent, non-adversarial Compliance Checker dedicated solely to categorical eligibility guidelines and institutional rules.

Heavy Computation vs. Practical Lightweight Orchestration: Many multi-agent frameworks require multi-turn recursive dialogue loops involving dozen-turn exchanges, incurring severe API latency and token costs that are impractical for standard web applications.

Lack of End-to-End Auditable Web Reference Architectures: There is a scarcity of lightweight, open implementations that integrate deterministic multi-agent prompt pipelines with full-stack web frameworks (such as Python Flask) to provide an accessible interface for human operators.

Addressing the Gap: This project directly bridges these gaps by engineering a streamlined, deterministic, four-stage multi-agent pipeline implemented within a clean Flask web interface. By incorporating an independent Compliance Checker alongside the Advocate and Skeptic prior to final adjudication, the architecture isolates institutional policy verification from adversarial debate, delivering an auditable, multi-perspective decision-support trail suitable for practical administrative screening.

CHAPTER 3

EXISTING SYSTEM

3.1 Traditional Systems

Automated and semi-automated decision-making in administrative, financial, and educational domains has historically relied on three main paradigms: manual human review, rule-based expert systems, and single-model machine learning classifiers.

3.1.1 Manual Decision-Making

Manual human review constitutes the traditional benchmark for institutional evaluations. Applications for loans or university admissions are assigned to human officers who examine submitted documentation, verify records, and render subjective assessments. While human judgment inherently accommodates qualitative nuance, empathy, and holistic context, manual decision-making exhibits critical structural vulnerabilities: severe throughput bottlenecks, substantial operational latency, high organizational labor expenses, and unavoidable intra-reviewer and inter-reviewer inconsistency, where identical profiles receive divergent outcomes depending on individual reviewer fatigue or subjective bias.

3.1.2 Rule-Based Decision Systems

To achieve operational scalability, organizations implemented rule-based expert systems and deterministic decision trees. These systems evaluate applications against programmed boolean thresholds (e.g., checking if credit scores or GPAs exceed rigid numerical cutoffs). While rule-based systems deliver perfect execution determinism and immediate auditability, their fatal limitation is absolute brittleness. They cannot evaluate qualitative narrative text, such as applicant personal statements or explanations of transient financial anomalies, leading to the categorical rejection of qualified non-traditional applicants.

3.1.3 Single Machine-Learning Models

The introduction of supervised statistical classifiers (Logistic Regression, Support Vector Machines, Random Forests, XGBoost, and Deep Neural Networks) enabled organizations to predict applicant default risks or academic success probabilities based on historical training datasets. Although effective at capturing complex non-linear correlations, statistical models function as opaque black boxes. Post-hoc explainability methods like SHAP or LIME offer local feature-importance scores but fail to provide human-interpretable causal narratives. Furthermore, statistical models are vulnerable to perpetuating historical prejudices embedded in training corpora.

3.1.4 Conventional Automated Screening

Contemporary automated screening frequently blends hard rule filtering with single-model scoring. In modern workflows, monolithic Large Language Model prompts are sometimes deployed to ingest application text and generate immediate recommendations. However, prompting a single LLM instance in a single pass results in unified, un-debated outputs where the model's initial semantic association dictates both the decision and the subsequent explanatory justification.

3.1.5 Limitations of Existing Systems

The key limitations across conventional decision-support frameworks can be summarized as follows:

Single-Perspective Decisions: Existing automated systems process inputs through a single evaluative lens, making them vulnerable to confirmation bias and premature convergence.

Limited and Post-Hoc Explanations: Explanations generated by single-pass systems represent rationalizations constructed to defend a predetermined score rather than an audit of competing factors.

Inflexible Rules: Rigid thresholding systems lack the capacity to interpret qualitative narrative nuances or weigh holistic mitigating factors.

Difficulty Handling Complex or Conflicting Evidence: When applicant profiles present contradictory indicators, single systems lack the dialectical mechanism to systematically debate these trade-offs.

Challenges in Policy and Compliance Checking: Conventional models frequently blur strict regulatory eligibility thresholds with qualitative merits, resulting in either unlawful policy breaches or unjust disqualifications.

CHAPTER 4

PROPOSED SYSTEM

4.1 Overview

The Multi-Agent LLM-Based Transparent Decision-Making System addresses the deficiencies of single-perspective models by structuring decision support as a sequential, multi-agent committee deliberation. Built as a full-stack Python Flask web application integrating the Google Gemini API, the system divides applicant evaluation into specialized cognitive roles: an Advocate Agent, a Skeptic Agent, a Compliance Checker Agent, and an Adjudicator Agent. Rather than training private foundation models, the system employs advanced prompt engineering, deterministic prompt chaining, and structured context aggregation. The application currently demonstrates evaluation capabilities across two operational modules: Loan Pre-Screening Evaluation and Academic Admissions Support.

4.2 System Architecture

The architecture decouples the presentation layer, the application control layer, and the external inference provider into a clean three-tier architecture (Figure 4.1).

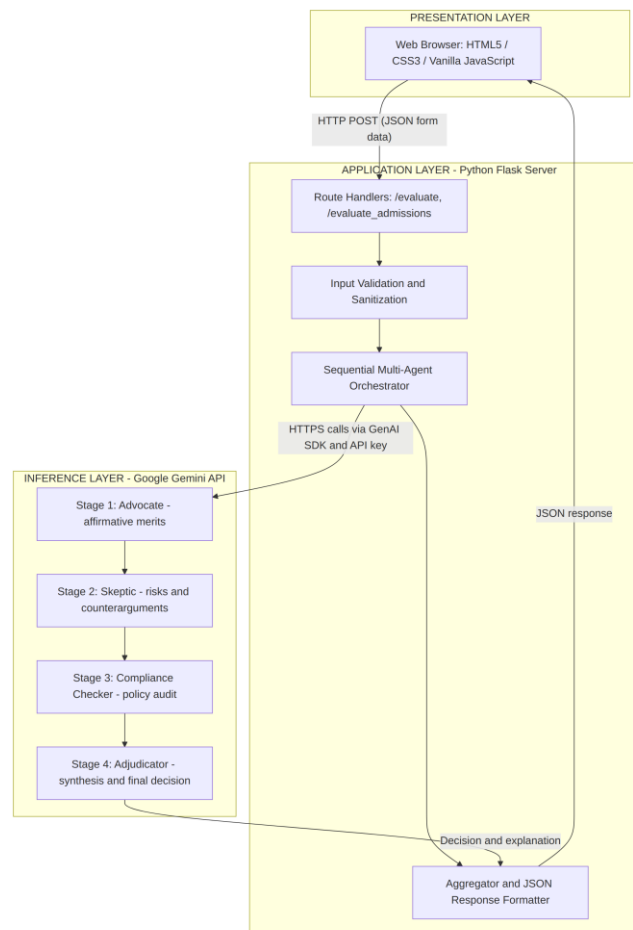


Figure 4.1: Three-tier system architecture

4.3 Agent Roles and Prompt Engineering

Each agent in the pipeline is conditioned using targeted system prompts designed to enforce strict behavioral boundaries:

4.3.1 Advocate Agent

The Advocate Agent is conditioned to extract and articulate the strongest affirmative arguments supporting approval. It analyzes applicant financial assets, income stability, career trajectory, academic achievements, research publications, or leadership qualifications, presenting a dedicated affirmative case without generating risk criticisms.

4.3.2 Skeptic Agent

The Skeptic Agent acts as an adversarial risk auditor. It scrutinizes the application for liabilities, debt-to-income vulnerabilities, credit anomalies, academic grade deficits, unverified claims, or institutional risks. Its directive is to challenge affirmative assumptions and identify reasons counseling caution or rejection.

4.3.3 Compliance Checker Agent

The Compliance Checker Agent operates objectively and non-adversarially. It evaluates the application against pre-established regulatory boundaries, institutional criteria, minimum income or credit thresholds, prerequisite degree accreditations, or statutory lending guidelines, categorizing elements as strictly compliant, non-compliant, or conditionally eligible.

4.3.4 Adjudicator Agent

The Adjudicator Agent functions as the presiding judicial evaluator. It receives the raw application data alongside the compiled briefs from the Advocate, Skeptic, and Compliance Checker. It weighs the conflicting arguments, reconciles trade-offs, and synthesizes a final binary decision (APPROVED or REJECTED) accompanied by an auditable, multi-perspective explanation.

4.4 System Workflow

The execution sequence follows a strict deterministic workflow: (1) User inputs applicant data via web form. (2) Flask backend validates data completeness. (3) Backend invokes Advocate Agent via Gemini API. (4) Backend invokes Skeptic Agent via Gemini API. (5) Backend invokes Compliance Checker Agent via Gemini API. (6) Backend supplies all prior outputs to Adjudicator Agent. (7) Adjudicator outputs binary decision and explanation. (8) Flask renders all four agent analyses on the web interface.

4.5 Decision and Explanation Generation

The final output produced by the Adjudicator Agent consists of an explicit binary determination (APPROVED or REJECTED) accompanied by a balanced natural-language synthesis. The explanation directly cites the affirmative merits recognized by the Advocate, the risk liabilities highlighted by the Skeptic, and the policy checks confirmed by the Compliance Checker, detailing exactly why the winning perspective prevailed.

4.6 Advantages and Limitations

Advantages: Provides procedural transparency, decoupled compliance auditing, active risk surfacing, modular extensibility across domains, and accessible human-readable reporting.

Limitations: Cumulative network latency resulting from four sequential API calls per evaluation, complete operational dependence on Google Gemini API availability, and stateless execution without persistent database storage [\[CONFIRM FROM SOURCE CODE\]](#).

CHAPTER 5

SYSTEM DESIGN

5.1 Class Diagram

The Class Diagram specifies the structural design, controller classes, data encapsulation models, and API interfaces.

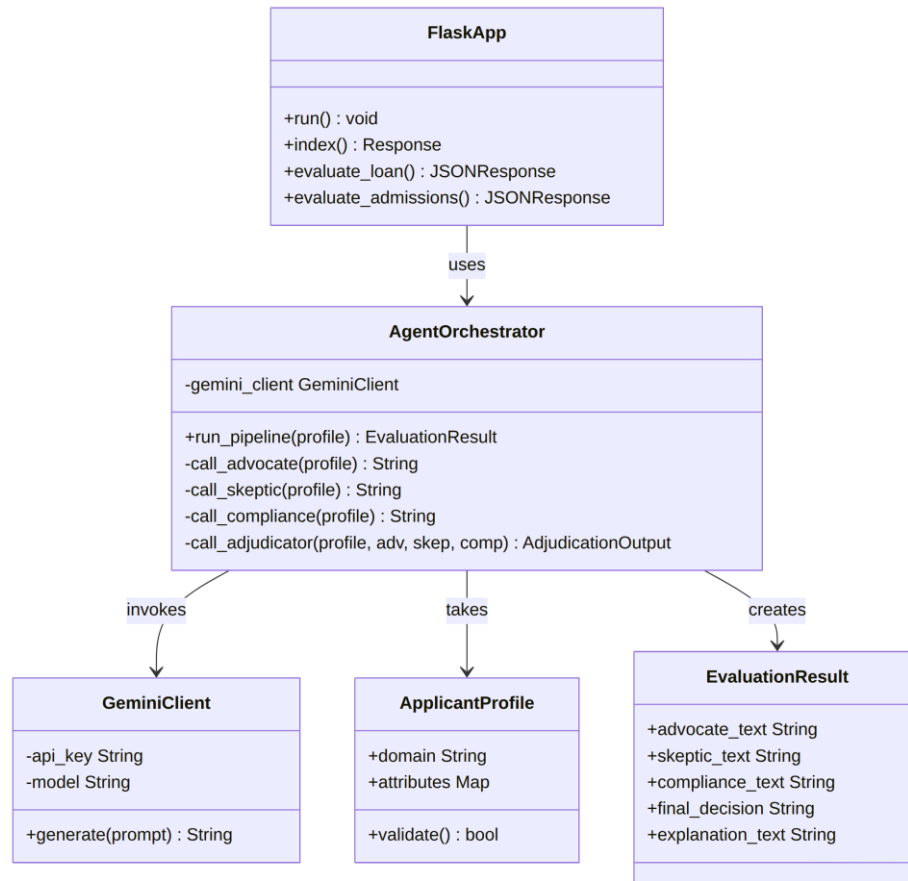


Figure 5.1: Class diagram

Description: The **FlaskApp** handles HTTP requests and delegates processing to the **AgentOrchestrator**. The orchestrator validates applicant data via **ApplicantProfile**, sequentially queries the **GeminiClient** across four agent stages, and returns an aggregated **EvaluationResult**.

5.2 Use Case Diagram

The Use Case Diagram defines the interactions between the administrative user (reviewer) and the decision-support system.

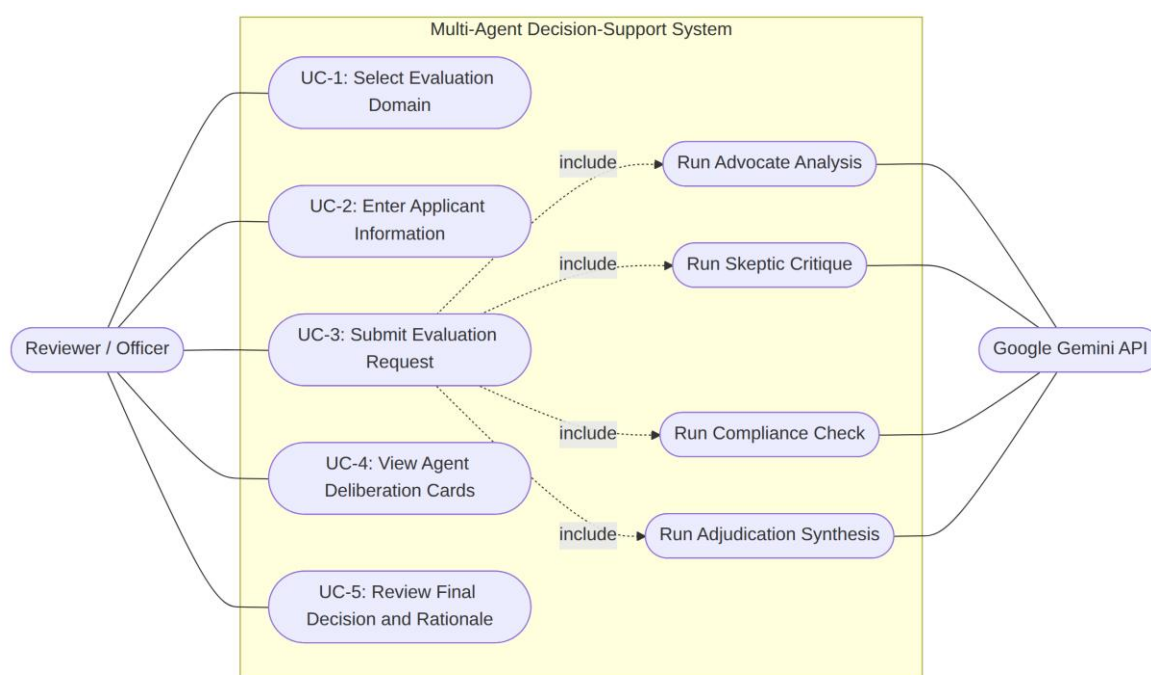


Figure 5.2: Use case diagram

Description: The primary human actor interacts with the system to select the evaluation domain, input applicant attributes, trigger the multi-agent pipeline, and examine the resulting multi-perspective deliberation audit trail.

5.3 Sequence Diagram

The Sequence Diagram details the chronological message exchanges between system components during an evaluation cycle.

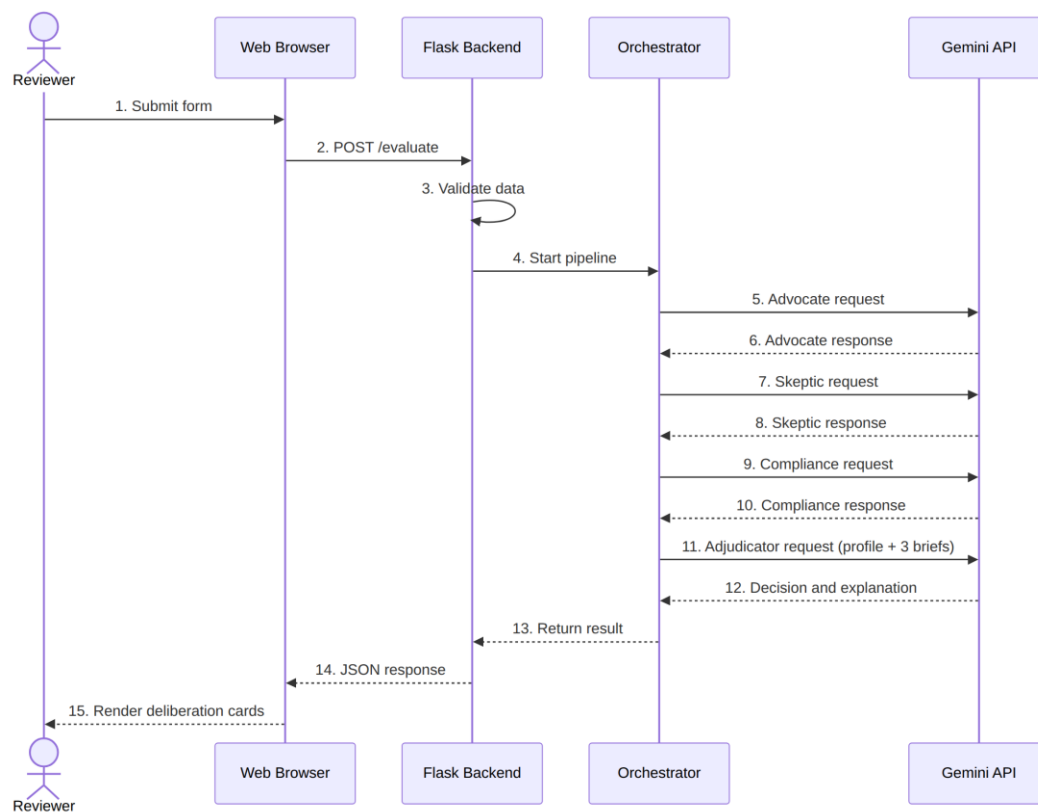


Figure 5.3: Sequence diagram

Description: Highlights the deterministic, sequential communication between the Flask backend, internal orchestrator, and external Google Gemini API across the four inference passes.

5.4 Component Diagram

The Component Diagram depicts the functional software modules and their structural relationships.

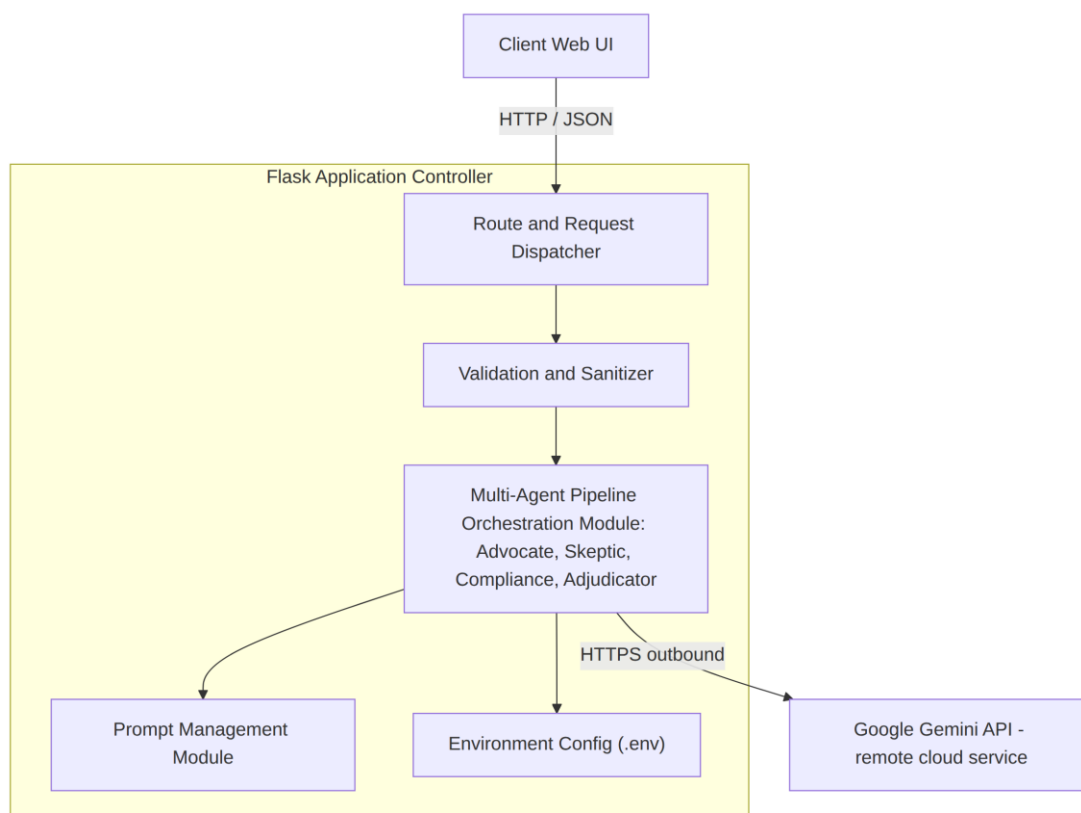


Figure 5.4: Component diagram

Description: Illustrates high cohesion and loose coupling between routing, validation, prompt generation, orchestration, and external API communication.

5.5 Deployment Diagram

The Deployment Diagram illustrates the physical and execution environment of the application.

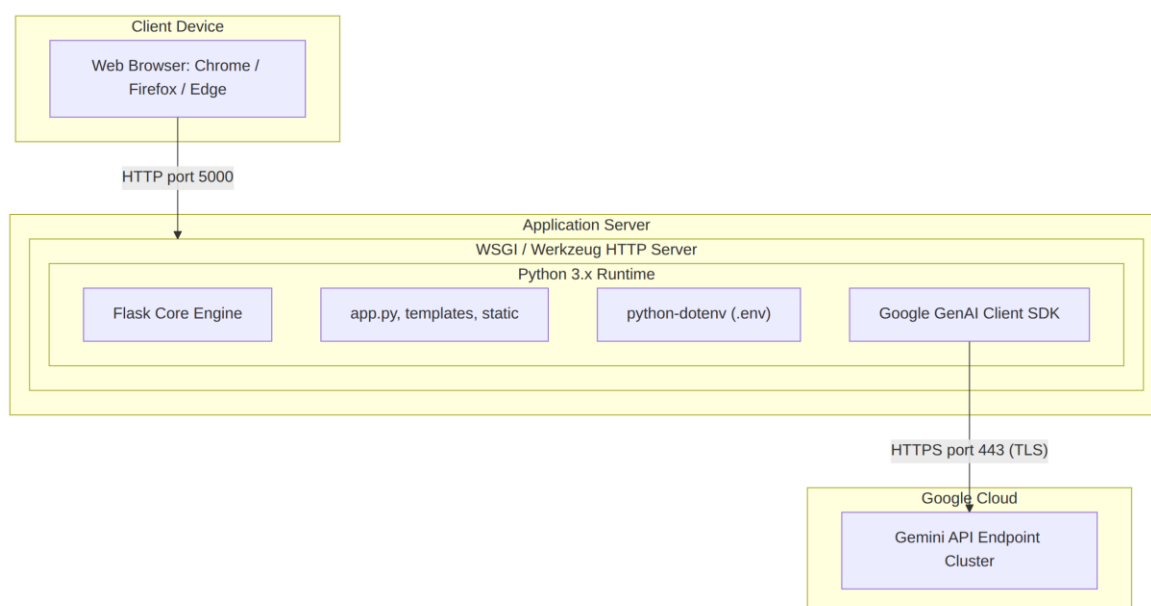


Figure 5.5: Deployment diagram

Description: Represents the client browser interacting with the local or hosted Flask server over HTTP, which connects to the Google cloud infrastructure via secure HTTPS.

5.6 Activity Diagram

The Activity Diagram describes the operational and control flow during evaluation processing.

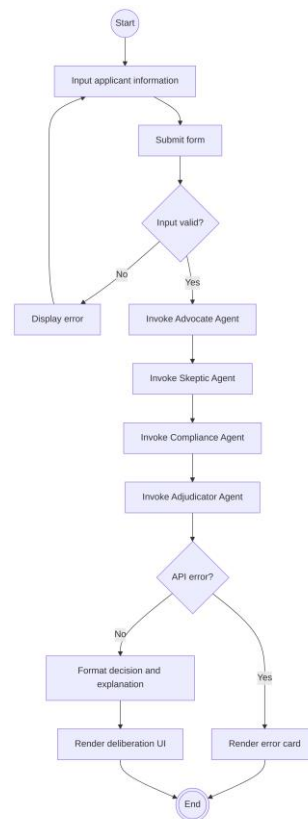


Figure 5.6: Activity diagram

Description: Illustrates decision points, validation branching, sequential agent execution, and error handling paths.

5.7 Data Flow Diagrams (DFD)

5.7.1 DFD Level 0 (Context Level Diagram)

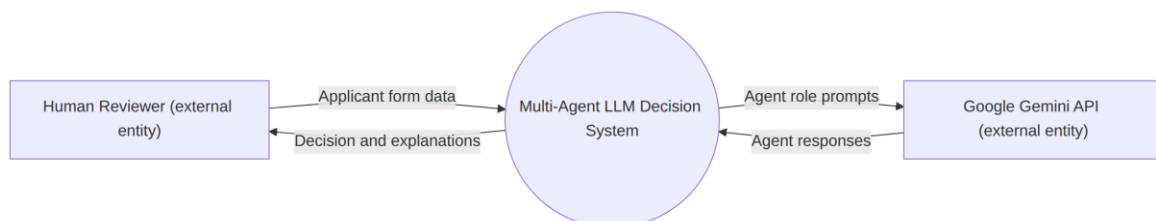


Figure 5.7: DFD Level 0 (context diagram)

Description: Outlines the external entities (Human Reviewer and Google Gemini API) interacting across the system boundary.

5.7.2 DFD Level 1 (Decomposed Process Diagram)

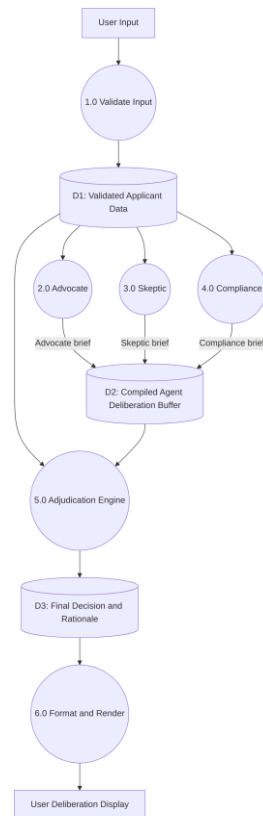


Figure 5.8: DFD Level 1 (decomposed process diagram)

Description: Traces data transformations through validation, three sequential agent analyses, buffer aggregation, adjudication synthesis, and client output rendering.

CHAPTER 6

SYSTEM REQUIREMENTS

6.1 Software Requirements

The software architecture relies exclusively on lightweight, standardized open-source components and cloud API integrations. The requirements are categorized into runtime environments, framework dependencies, and development tools:

Operating System: Platform independent; compatible with Windows 10/11, Ubuntu 20.04/22.04 LTS, or macOS 12+ [**CONFIRM FROM PROJECT**].

Programming Language: Python 3.9+ (Python 3.10 / 3.11 recommended).

Web Microframework: Flask (Python WSGI web application framework).

Generative AI Client SDK: google-generativeai (Official Python client library for the Google Gemini API).

Configuration Management: python-dotenv (for loading environment variables from .env files).

Frontend Technology Stack: HTML5, CSS3, Modern JavaScript (Fetch API for asynchronous JSON requests).

Data Interchange Standard: JSON (JavaScript Object Notation).

Development Tools: Visual Studio Code (VS Code) or compatible IDE, Git version control.

Client Web Browser: Google Chrome, Mozilla Firefox, Microsoft Edge, or Safari with JavaScript enabled.

Authentication Key: Valid Google Gemini API Key configured in the environment file.

6.2 Hardware Requirements

Because all intensive Large Language Model computations are handled offsite by Google's cloud infrastructure, the physical hardware requirements for hosting and accessing the application are minimal:

6.2.1 Development and Host Workstation (Minimum)

Processor (CPU): Dual-Core x86_64 or ARM64 processor (Intel Core i3, AMD Ryzen 3, or Apple M-series equivalent).

System Memory (RAM): 4 GB RAM minimum (8 GB recommended for concurrent development tasks).

Storage Capacity: 500 MB of available secondary storage for Python runtime, dependencies, and application source files.

Network Interface Card: Active Internet connectivity (broadband or Wi-Fi) with outbound HTTPS access on TCP Port 443 to communicate with the Google Gemini API.

6.2.2 Client Workstation (End-User Interface)

Processor: Standard modern desktop, laptop, or tablet processor capable of running a modern browser.

System Memory: 2 GB RAM minimum.

Display Resolution: 1280 x 720 minimum (1920 x 1080 Full HD recommended for optimal multi-column deliberation card viewing).

CHAPTER 7

SYSTEM REQUIREMENTS SPECIFICATION

7.1 Functional Requirements

Functional requirements define the core operational features, data handling capabilities, and interactive workflows supported by the application. The verified functional requirements are listed below:

FR-01 – Applicant Data Intake: The web interface must provide form input fields for capturing applicant information (financial or academic profile data). **(Status: Implemented)**

FR-02 – Domain Selection: The application must support domain-specific evaluation modes, specifically Loan Pre-Screening and Academic Admissions. **(Status: Implemented)**

FR-03 – Input Validation: The backend controller must validate that mandatory fields are non-empty and conform to expected data types prior to initiating API calls. **(Status: Implemented)**

FR-04 – Error Notification: If submitted input data is incomplete, malformed, or invalid, the system must abort processing and display an informative error message. **(Status: Implemented)**

FR-05 – Advocate Agent Invocation: The system must construct an affirmative evaluation prompt containing applicant data and invoke the Google Gemini API to extract strengths and merits. **(Status: Implemented)**

FR-06 – Skeptic Agent Invocation: The system must construct an adversarial risk audit prompt and invoke the Google Gemini API to identify liabilities, risks, and counterarguments. **(Status: Implemented)**

FR-07 – Compliance Agent Invocation: The system must formulate a compliance prompt to verify applicant attributes against specified eligibility rules and policy thresholds. **(Status: Implemented)**

FR-08 – Adjudicator Deliberation: The system must combine applicant data, Advocate output, Skeptic output, and Compliance output into an adjudicative prompt and invoke Gemini for final deliberation. **(Status: Implemented)**

FR-09 – Binary Decision Generation: The Adjudicator Agent must output an explicit binary determination conforming to APPROVED or REJECTED. **(Status: Implemented)**

FR-10 – Explanatory Rationale Synthesis: The system must produce a coherent, structured natural-language rationale synthesizing the arguments across the multi-agent deliberation. **(Status: Implemented)**

FR-11 – API Exception Handling: The application must handle upstream API timeouts, network disconnections, invalid keys, or quota exhaustion gracefully without crashing the server. **(Status: Implemented)**

FR-12 – Client Result Rendering: The web frontend must dynamically render the binary verdict alongside the individual Advocate, Skeptic, and Compliance findings into accessible UI cards. **(Status: Implemented)**

FR-13 – Persistent Storage & Logging: Storing applicant evaluation history, audit trails, and decisions in a local or cloud database for historical review. **(Status: [CONFIRM FROM SOURCE CODE / Unimplemented])**

FR-14 – Role-Based Access Control: User authentication, administrative roles, and access tier verification. **(Status: [CONFIRM FROM SOURCE CODE / Unimplemented])**

7.2 Non-Functional Requirements

Non-functional requirements specify quality benchmarks, operational constraints, and architectural characteristics:

Performance: Local Flask request handling latency is negligible (< 50 ms). End-to-end response time is primarily governed by the cumulative duration of the four sequential round-trip Gemini API calls (typically **[INSERT ACTUAL VALUE]** to **[INSERT ACTUAL VALUE]** seconds depending on network conditions).

Reliability & Robustness: The system employs defensive exception handling around external HTTP calls to ensure network disconnects, rate limits, or invalid API responses do not result in unhandled HTTP 500 server crashes.

Security: API credentials are kept strictly out of client-side code and public version control, loaded into memory via environment variables (.env). Input fields are sanitized to avoid prompt formatting errors.

Usability: The interface presents affirmative, skeptical, compliance, and adjudicative outputs in distinct visual cards, enabling fast cognitive scanning by administrative officers.

Scalability: The stateless nature of the Flask backend allows straightforward containerization and horizontal scaling behind reverse proxies, although concurrency is ultimately bounded by the user's Google Gemini API account rate limits.

Maintainability: Clear modular separation between web routes (app.py), prompt templates, and presentation assets ensures that agent roles and criteria can be updated without altering the core routing engine.

Availability: Highly available locally, but end-to-end decision support is bound to external Google Gemini API service uptime.